Statistical learning from nonrecurrent experience with discrete input variables and Recursive Error Minimization (REM) equations

Jeffrey R. Carter and Wayne E. Simon

Martin Marietta Astronautics Group
MS DC4460
P. O. Box 179
Denver, CO 80201

ABSTRACT

Neural networks are trained using Recursive Error Minimization (REM) equations to perform statistical classification. Using REM equations with continuous input variables reduces the required number of training experiences by factors of one to two orders of magnitude over standard back propagation. Replacing the continuous input variables with discrete binary representations reduces the number of connections by a factor proportional to the number of variables, reducing the required number of experiences by another order of magnitude.

Undesirable effects of using recurrent experience to train neural networks for statistical classification problems are demonstrated and nonrecurrent experience used to avoid these undesirable effects.

1. THE I-4I PROBLEM

The statistical classification problem which we address is is that of assigning points in d-dimensional space to one of two classes. The first class has a covariance matrix of I (the identity matrix); the covariance matrix of the second class is 4I. For this reason the problem is known as the I-4I problem. Both classes have equal probability of occurrence, and samples from both classes may appear anywhere throughout the d-dimensional space. Most samples near the origin of the coordinate system will be from the first class, while most samples away from the origin will be from the second class. Since the two classes completely overlap, it is impossible to have a classifier with zero error. The minimum possible error is known as the Bayes error and can be computed. The Bayes classifier is a hypersphere with a radius equal to the square root of $1.85d$. Samples inside the hypersphere are classified as class I, and samples outside it are class II.

Hush has examined training neural networks to perform classification for the I-4I problem using standard back propagation and continuous input variables[1]. Note that the data for this problem is inconsistent; this is what makes the problem interesting and the feature it has in common with the real world. Hush trained neural networks on inconsistent data by selecting a finite training set and repeatedly presenting the training set to the network. Hush used training sets ranging from $50d$ to $400d$ samples per class. Networks with a sufficient number of connections can learn the topology of the training set very closely. Since the ultimate desired output of the network is a generalization of the training which approximates the hypersphere classifier, such recurrent training produces generalizations which are too specific to the training set. Such networks can classify the training set with nearly zero error, but perform less well on

independent test data. This effect is clearly visible in Hush's Figures 4 and 6, which show errors very near zero for the training sets, but much higher error rates on a separate test set.

We avoid this problem by using nonrecurrent training sets. This means that each sample in the training set is presented to the network only once. Such a training set must be much larger than one used for recurrent training. We did not actually generate a fixed training set, but used a long-period random number generator[2] to produce experiences "on the fly." The random number generator we used allows the seeds to be set explicitly, which allowed us to duplicate sequences of numbers when desired.

2. REM EQUATIONS AND CONTINUOUS INPUT VARIABLES

We will not present the Recursive Error Minimization (REM) learning equations in this paper. Interested readers should refer to our previous papers on REM equations[3,4]. Note that the technique which guarantees convergence to a global minimum assumes that the problem has a consistent, finite input space and that recurrent training will be used. This is not the case for the I-4I problem, but if the initial weights of the network are sufficiently small, the output of the network is nearly the same regardless of its inputs, and this technique can be used.

We first applied REM equations to duplicating Hush's work, using continuous input variables for the four-, eight-, and twelve-dimensional versions of the I-4I problem. Hush determined that a neural network must have at least d + 1 intermediate nodes, and best results are obtained with $3d$ intermediate nodes. Using more than $3d$ intermediate nodes does not improve the network's performance. We therefore used $3d$ intermediate nodes for our networks.

For consistency with discrete input variable versions of the same problem, we limited the magnitude of the samples to be in the range -3.999 to +3.999. Values outside this range were set to -3.999 or +3.999, depending on the sign.

Hush ran all of his training sessions for 200,000 experiences, regardless of the size of the network. We found through experimentation that REM equations required $75c$ or fewer training experiences for the level of error to stabilize, where c is the number of connections in the network. For four dimensions, we trained for 7,300 experiences (c = 73), for eight dimensions, 24,100 experiences (c = 241), and for twelve dimensions, 50,500 experiences (c = 505). REM equations reduced the number of training experiences by one to two orders of magnitude, as we expected from our earlier work.

We tested the resulting networks on 10,000 samples per class of independent test data. The four-dimensional version had an error rate of 21.35%±0.29%, the eight-dimensional, 12.88%±0.26%, and the twelve-dimensional, 9.59%±0.16%. These are 3.75%, 3.88%, and 4.69% above the Bayes error for these problems, respectively. Hush only presents results for independent test data in graphical form. Reading values of error for independent test data and $3d$ intermediate nodes from his Figures 1, 2, and 3 produces values of approximately 19% for the four-dimensional problem, 12% for the eight-dimensional problem, and 7.8% for the twelve-dimensional problem. Simply using REM equations on this problem produced equivalent results after significantly fewer training experiences.

3. REM EQUATIONS AND DISCRETE INPUT VARIABLES

Input variables were made discrete by representing them as signed binary numbers. Values were first limited to the range -3.999 to +3.999. Values outside this range were set to -3.999 or +3.999, depending on the sign. The sign bit was determined and the magnitude of the value was converted to an unsigned binary number in the standard manner.

We experimented with various numbers of bits of precision. The minimum number of bits was two, the sign bit and one magnitude bit. We used the convention that the most significant magnitude bit always represented 2^1, regardless of the number of magnitude bits being used. This is why inputs were limited to a magnitude of +3.999. Additional bits represented 2^0, 2^{-1}, 2^{-2}, and 2^{-3}, respectively.

The resulting binary number was used to set the output values of input nodes of the network. Nodes corresponding to bits with the value one were set to +1, and nodes corresponding to zero bits were set to -1.

There are a number of consequences of making the input variables discrete. First, the number of input nodes changes from d to bd, where b is the number of bits of precision being used. Second, there are two representations for zero, +0 and -0. These should be interpreted as +0 meaning a value greater than or equal to zero but less than the smallest non-zero unsigned binary value, and -0 meaning less than zero but greater than the negative of the smallest non-zero unsigned binary value.

Third, another effect of making the input discrete is that the coupling between experiences with different values of input is reduced. This simplifies the learning process, but makes it desirable to remember longer, as experience proceeds, to achieve the advantage in precision made possible by discrete input. REM equations use a parameter, P, which is the number of following experiences necessary to reduce the memory of an experience by the factor e (2.718 . . .). For discrete input, P is started with a value proportional to the number of connections in the network. Then, as experience accumulates, P is made proportional to the total number of experiences. Thus the memory of the network lengthens as the number of experiences increases. Experimentation has shown that this process does not improve the performance of a network using continuous input, and actually slows down the learning process.

We first experimented with various combinations of number of bits of precision and number of intermediate nodes (h) for the four-dimensional problem. Table I shows the levels of error for the various combinations tested.

These values were obtained using 1,000 samples per class of independent test data with the number of training experiences kept to a minimum. The Bayes error for this problem is 17.6%. There is no significant difference between any of the values in Table I. We chose the configuration with the largest number of connections (six bit input and four intermediate nodes for our subsequent tests. This is the configuration with the slowest learning.

When using discrete input variables we found that we needed no more than $50c$ experiences for the level of error to stabilize. We used 10,500 (c = 105), 20,100 (c = 201), and 29,700 (c = 297) experiences for the four-, eight-, and twelve-dimensional problems, respectively. The resulting error levels were 17.45%±0.22%, 9.84%±0.30%, and 5.56%±0.23%, respectively, using 10,000 samples per class of independent test data. The result for four dimensions is slightly below the Bayes error but within one standard deviation of it; the eight- and twelve-dimension results are 0.84% and 0.66% above the Bayes error.

Table I
Error Levels for the Four-Dimensional Problem.

h					
4	18.05±0.62	17.15±0.78	17.50±0.62	17.20±0.87	18.45±0.64
3	19.40±1.03	18.60±0.53	18.85±1.32	18.25±0.80	16.85±0.89
2	19.65±1.02	18.15±1.04	18.60±0.76	17.75±0.75	19.30±0.99
1	19.90±1.26	19.20±1.13	18.00±0.80	18.10±1.03	18.20±0.55
	2	3	4	5	6

b (Including Sign Bit)

These error rates are significantly better than any achieved by Hush or by us using continuous input variables. As Table I indicates, similar results could have been obtained using smaller values for b and h, with better results than Hush in two to three orders of magnitude fewer training experiences.

4. EFFECT OF NUMBER OF CONNECTIONS

We used large values for b and h; so large, in fact, that c, for the four-dimensional problem, was larger for the discrete case than for the continuous case. Even so, as the number of dimensions increases, the number of connections increases far more rapidly for continuous input variables than for discrete input variables. For continuous input variables with $h = 3d$, the number of connections for this problem is given by

$$c = 3d^2 + 6d + 1$$

while for discrete input variables it is given by

$$c = hbd + 2h + 1$$

Thus, for continuous input variables, c is a function of d^2, while, for discrete input variables, it is a function of d. For d = 1,000, which represents the kind of real-world problems for which neural networks should be suited, continuous input variables have c = 3,006,001, and discrete input variables, c = 24,009, two orders of magnitude less.

The number of calculations needed for a single experience is proportional to c. As we indicated above, the total number of training experiences to achieve a desired level of error is also proportional to c. Therefore, the total number of calculations is proportional to c^2. From the equations for c given above we can

see that the total number of calculations is proportional to d^2 for discrete input variables, and to d^4 for continuous input variables. For d = 1,000, a system with discrete input variables will need only one-millionth the calculations required for continuous input variables.

5. CONCLUSIONS

Training neural networks using discrete input variables with REM equations gives significantly better performance after significantly fewer training experiences, compared with training using continuous input variables and standard back propagation.

The use of discrete input variables also significantly reduces the number of connections in the network, and therefore the total number of calculations required to train the network. The orders of magnitude difference in the total number of training experiences required will be significant even for fully parallel networks.

6. REFERENCES

1. Hush, D., "Classification with Neural Networks: A Performance Analysis," *Proceedings of the IEEE International Conference on Systems Engineering*, IEEE, 1989.

2. Harmon, M., "An Ada Implementation of Marsaglia's 'Universal' Random Number Generator," *Ada Letters*, Vol. VIII, No. 2, 1988 Mar/Apr.

3. Simon, W., and J. Carter, "Back Propagation Learning Equations from the Minimization of Recursive Error," *Proceedings of the IEEE International Conference on Systems Engineering*, IEEE, 1989.

4. Simon, W., and J. Carter, "Learning to Identify Letters with REM Equations," *Proceedings of the International Joint Conference on Neural Networks*, Vol. I, Lawrence Erlbaum Associates, 1990.